

① (William中文第八版)P152页习题7.4

- 在DES的ECB模式中，若在密文的传输过程中，某一块发生了错误，则只有相应的明文分组会有影响，然而，在CBC模式中，这种错误具有扩散性，比如传输时 C_1 发生错误将会影响明文分组 P_1 和 P_2 。

a. P_2 以后的分组是否会受到影响？

- 假设 P_1 本来就有一位发生了错误，则这个错误要扩散多少个密文分组，对接受者解密后的结果有何影响？

a. 不会，因为解密时 C_1 引起的误差只能传递到 P_2 ($C_1 \oplus P_2$) 而不再继续向右传递： $P_i = D_K(C_i) \oplus C_{i-1}$

b. 会扩散到所有密文分组，因为 $C_i = E_K(C_{i-1} \oplus P_i)$

误差传递链： $P_i \rightarrow C_i \rightarrow C_{i+1} \rightarrow \dots \rightarrow C_N$

接受者解密后得到的明文分组($P'_1, P'_2, \dots, P'_N$)中只有 P_1 加错误被保留，其余分组都与原明文分组相同。因为

$$P'_i = D_K(C'_i) \oplus C'_{i-1}$$

$$= D_K(E_K(P_i \oplus C_{i-1})) \oplus C'_{i-1}$$

$$= P_i \oplus C_{i-1} \oplus C'_{i-1}$$

$$= P_i$$

② (William中文第八版)P153页习题7.6

- CBC-pad是RC5中的一种分组加密工作模式，它可以用于任何分组密码，并处理任意长度的明文，得到的密文最多比明文长一个分组的长度。填充的作用是保证输入明文是分组长度的整数倍。假设原始明文是整数字节，尾部填充1到 bb 个字节，这里 bb 为分组的字节长度。填充的字节相同，其值为填充的字节数。例如，若有8字节填充，则每个字节为00001000。那么为何不允许0字节的填充？若原始明文是分组大小的整数倍，为什么不省去填充？

统一添加 padding 便于自动进行数据处理。

如果填充 0x00，难以判断填充了多少字节；

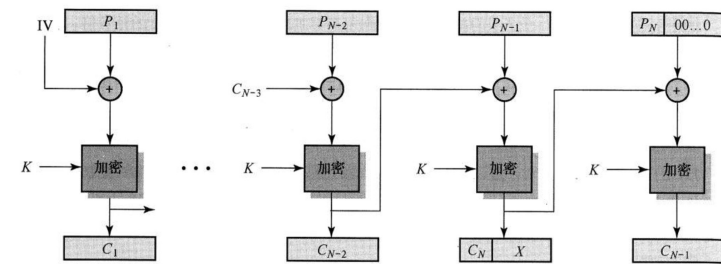
如果不做填充，则难以用统一的逻辑判断和处理填充与不填充两种情况。

③ (William中文第八版)P153页习题7.11

- 填充并不总是合适的，例如，我们希望使用相同的内存缓冲区（明文最初存储在这里）来存储加密的数据，这时密文必须与明文的长度相同。密文挪用模式（CTS）是满足这种要求的一种工作模式。图7.18 (a) 为该模式的实现过程。

a. 解释CTS是如何工作的；

b. 描述如何解密 C_{N-1} 和 C_N



a. 加密

$$C_i = E_K(P_i \oplus C_{i-1}), i = 1, 2, \dots, N-2, C_0 = IV$$

$$T_{N-1} = E_K(P_{N-1} \oplus C_{N-2})$$

$$C_{N-1} = E_K(P_N \parallel 00\dots 0 \oplus T_{N-1})$$

$$T_{N-1} = C_N \parallel X$$

b. 解密

$$P_i = D_K(C_i) \oplus C_{i-1}, i = 1, 2, \dots, N-2, C_0 = IV$$

$$P_{N-1} = D_K(T_{N-1}) \oplus C_{N-2} = D_K(C_N \parallel D_K(C_{N-1})[16\text{位}])$$

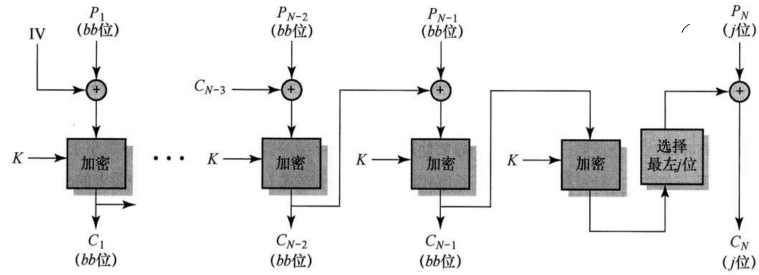
$$P_N = D_K(C_{N-1})[16\text{位}] \oplus C_{N-2}$$

④ (William中文第八版)P153页习题7.12

- 图7.18(b)中给出了一种CTS的替换方案，使得当明文不是分组长度整数倍时产生的密文长度与明文长度相等

a. 解释该算法

b. 解释为何CTS算法比图7.18(b)中的方法更可取？



a. 加密

$$C_i = E_K(P_i \oplus C_{i-1}), i = 1, 2, \dots, N-1, C_0 = IV$$

$$C_N = E_K(C_{N-1}[最左边j位] \oplus P_N)$$

解密

$$P_i = D_K(C_i) \oplus C_{i-1}, i = 1, 2, \dots, N-1$$

$$P_N = C_N \oplus E_K(C_{N-1}[最左边j位])$$

b. 该方法需要接收方知道加密算法且 P_N 并未被直接加密，没有经过加密函数的非线性变换，因此安全性较低。

- ⑤ 分组密码作用于 n 位明文分组，产生 n 位密文分组，当 $n=64$ 或者更大时，为什么使用 n 位分组的任意可逆代替密码不可行？

从 n 位明文分组到 n 位密文分组的映射有 $2^n! \sim \Theta(n2^n)$ 个，每个映射对应一个密钥，因此当 n 过大时难以存储密钥和对应的映射。